

# 计算机程序设计基础(2)

## --- C++程序设计(6)

孙甲松

[sunjiasong@tsinghua.edu.cn](mailto:sunjiasong@tsinghua.edu.cn)

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 62796193/13901216180

2022. 3.

1

## 5. 类的继承与派生

- 继承与派生(Inheritance and Derivation)是C++的重要机制，是面向对象程序设计的重要特征。
- 该机制自动为一个类提供来自另一个类的数据结构和操作。
- 这样可以利用已构造好的类生成新类，充分利用已有资源，进行代码复用（code reused）。
- 在构造新类的过程中保持已有类的特性称为继承。
- 在已有类的基础上新增自己的特性而构造新类的过程称为派生。

2

## 目录

- 5.1 概述
- 5.2 访问控制（继承性质）
- 5.3 保护成员
  - 5.3.1 派生类中成员的访问权限
- 5.4 友元与继承
- 5.5 访问权限调整
- 5.6 成员名限定
- 5.7 派生类的构造函数与复制构造函数
- 5.8 带基类内嵌对象成员的派生类
- 5.9 动态束定与虚函数
  - 5.9.1 关于类型兼容性原则
- 5.10 纯虚函数
- 5.11 虚析构函数
- 5.12 多继承
- 5.13 多继承的二义性
- 5.14 虚基类

3

## 5.1 概述

类继承的语法形式:

```
class 标识符2 : 访问控制 标识符1
{ 新成员和成员函数
};
```

其中:

标识符1 ----- 基类

标识符2 ----- 派生类

访问控制 ----- 类的继承方式，有三种:

public: 公有继承

protected: 保护继承

private: 私有继承

新成员和成员函数: 指在继承基类的成员和成员函数后，派生类新增加的成员和成员函数。

4

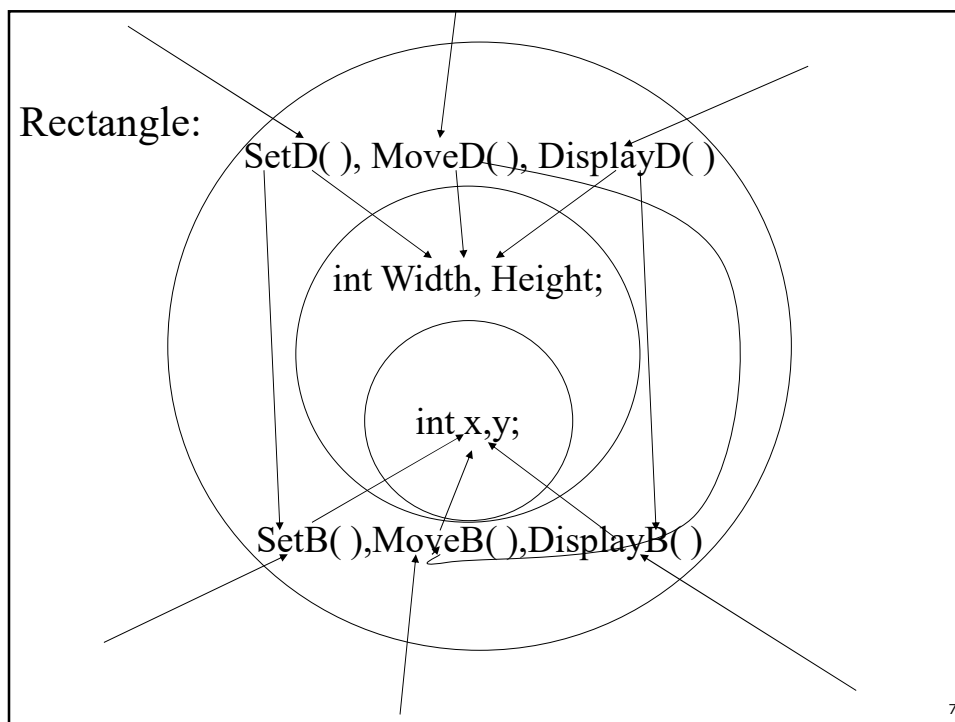
例如: 点类与矩形类

```
class Shape {  
    public: void SetB(int x, int y);  
           void MoveB(int x, int y);  
           void DisplayB( ) const;  
    private: int X, Y;  
};  
class Rectangle: public Shape {    // 公有继承  
    public: void SetD(int x, int y, int w, int h);  
           void MoveD(int x, int y );  
           void DisplayD( ) const;  
    private: int Width, Height;  
};
```

5

- 说明:
- Shape是一个点类， 有：  
两个私有成员： X， Y， 表示平面上一个点的坐标(X, Y);  
三个成员函数: SetB( ), MoveB( ), DisplayB( )
- Rectangle是矩形类， 是Shape类的派生类， 有：  
四个私有成员： X， Y， Width, Height  
六个成员函数: SetB( ), MoveB( ), DisplayB( ), SetD( ), MoveD( ), DisplayD( )
- Rectangle类公有继承了基类Shape类的全部成员和成员函数， 并加入了新的成员和成员函数。

6



## 5.2 访问控制（继承性质）

- ◆ 当继承性质为**public**（公有继承），则派生类全部公有继承基类的成员和成员函数，访问权限保持不变。派生类外和派生类的成员函数只能访问基类中具有公有访问特性的成员或成员函数。
- ◆ 当继承性质为**private**（私有继承），则基类的任何性质的成员或成员函数都变成了派生类的私有成员或成员函数，除了派生类的成员函数可以直接访问其基类的公有和保护成员(函数)外，禁止派生类外对任何性质(包括公有)的基类成员或成员函数的直接访问。
- ◆ 通过不同性质的继承，派生类改变了对基类的成员和成员函数的访问权限。

8

## 访问权限的继承和调整

| 继承性质      | 基类中成员(函数)的访问权限                 | 继承后基类中的成员(函数)在派生类中访问权限            |
|-----------|--------------------------------|-----------------------------------|
| public    | public<br>protected<br>private | public<br>protected<br>private    |
| protected | public<br>protected<br>private | protected<br>protected<br>private |
| private   | public<br>protected<br>private | private<br>private<br>private     |

9

例如5.1节的类中：

```
void Shape::SetB(int x, int y)
{ X=x; Y=y; }
void Rectangle::SetD(int x, int y, int w, int h)
{   SetB(x, y); // 不可以 X=x, Y=y;
    Width = w;
    Height = h;
}
```

虽然Rectangle类继承了Shape类的所有成员和成员函数，Shape类的私有成员X，Y也是Rectangle类的私有成员，但Rectangle类的成员函数SetD()仍不能直接访问X，Y，而需要通过基类Shape的公有成员函数SetB()间接访问。

10

```

#include <iostream>
using namespace std;
class Shape {
    public: void SetB(int x, int y);
           void MoveB(int x, int y);
           void DisplayB( ) const; // 常函数
    private: int X, Y;
};
class Rectangle: public Shape { // 公有继承
    public: void SetD(int x, int y, int w, int h);
           void MoveD(int x, int y);
           void DisplayD( ) const; // 常函数
    private: int Width, Height;
};
void Shape::SetB(int x, int y)
{ X=x; Y=y; }
void Shape::MoveB(int x, int y)
{ X += x; Y += y; }

```

11

```

void Shape::DisplayB( ) const
{ cout << X << ' ' << Y << endl; }
void Rectangle::SetD(int x, int y, int w, int h)
{ SetB(x, y); // 不可以 X=x, Y=y;
  Width = w; Height = h;
}
void Rectangle::MoveD(int x, int y)
{ MoveB(x, y); // 不可以 X += x, Y += y;
}
void Rectangle::DisplayD( ) const
{ DisplayB( ); // 不可以 cout << X << ' ' << Y << endl;
  cout << ' ' << Width << ' ' << Height << endl;
}
void main( )
{ Rectangle R; R.SetD(5, 10, 12, 14);
  Shape S; S.SetB(2, 4);
  R.DisplayD( ); S.DisplayB( );
}

```

运行结果:

```

5 10
12 14
2 4

```

12

```

#include <iostream> // 私有方式继承的例子
using namespace std;
class A {
public:
    A(int a=0,int b=0) : a(a),b(b){ }
    void print() { cout << a << endl; }
    int b; // b为A类的公有成员
private:
    int a; // a为A类的私有成员
};
class B : private A { // B类以私有方式继承A类
public:
    B(int a, int b, int c) : A(a, b),c(c){ }
    void print()
    { cout << c << endl; //成员函数可直接访问本类的私有成员
      cout << b << endl; //成员函数可直接访问基类的公有成员
      A::print(); //成员函数不可以直接访问基类的私有成员a
      // 若直接写 cout << a << endl; 编译将出现错误:
    } // error C2248: "A::a": 无法访问private 成员(在“A”类中声明)
private:
    int c; // c为B类的私有成员
};

```

13

```

class C : public B { // C类以公有方式继承B类
public:
    C(int a, int b, int c, int d) : B(a, b,c),d(d){ }
    void print()
    { cout << d << endl;
      B::print(); // 调用基类B的成员函数间接打印c,b,a
      // 若直接写 cout << c << endl; 编译将出现错误:
      // error C2248: "B::c": 无法访问private 成员(在“B”类中声明)
      // 若直接写 cout << b << endl; 编译将出现错误:
      // error C2247: "A::b"不可访问, 因为“B”使用“private”从“A”继承
      // 用基类B的基类A的公有成员函数 A::print(); 打印, 编译将出现错误:
      // error C2247: "A::print"不可访问, 因为 “B”使用 “private”从 “A”继承
    }
private:
    int d; // d为C类的私有成员
};
void main()
{ C c(1,2,3,4);
  c.print();
  c.B::print(); // 调用基类B的print函数
}

```

运行结果:

```

4
3
2
1
3
2
1
请按任意键继续...

```

14

## 5.3 保护成员

用protected声明的成员被称为保护成员。例如：

```
#include <iostream>
using namespace std;
class MyClass {
public:    void Print( ){cout<<X<<" "<<Y<<" "<<val; }
        void Set(int x, int y, int v){X=x; Y=y; val=v; }
        int p;
protected: int val;
private:   int X, Y;
};
void main( )
{ MyClass My;
  My.Set(3,4,5);
  My.p=5;      // 公有成员，可以直接访问
  My.val = 10; // 编译错误，不允许类外直接访问，只有成员函数可以
               // error C2248: "MyClass::val": 无法访问 protected 成员(在 "MyClass"类中声明)
  My.Print( );
}
```

15

## 5.3 保护成员(续1)<sup>JS1</sup>

保护成员具有双重身份：

- 对于建立它的类，它与private成员的性质相似。
- 对于继承此类建立的类，它与public成员性质相似。

```
class YourClass : public MyClass
{ public: void Update(int x, int y, int v);
};
void YourClass::Update(int x, int y, int v)
{ Set(x, y, v);
  val = v*2; // YourClass类可以直接访问val, 但不能直接访问X,Y
}
void main( )
{ YourClass You;
  You.Update(3,5,7);
  You.Print( ); // 调用基类的Print函数打印
  You.val = 10; // 不允许直接访问val，只有YourClass类的成员函数可以
               // error C2248: "MyClass::val": 无法访问 protected 成员(在 "MyClass"类中声明)
```

删掉You.val = 10;语句后，运行结果：  
3 5 14请按任意键继续...

16





## 5.3 保护成员(续2)

需要注意的是：

在main函数中不能出现 `You.val=10;` 说明  
`val`仍不是Yourclass真正的public成员，只  
是Yourclass的protected成员，算Yourclass  
类的private成员。

归纳一下：

17

## 5.3 保护成员(续3)

若A是基类，B是基于A的public派生类，则：

- 基类A的private成员(函数)，成为派生类B的private成员(函数)，但派生类B的成员函数不能直接访问基类A的私有成员(函数)，而只能通过基类A的成员函数间接访问，它们仍然是基类A的private成员，但基类A是派生类B的一部分。
- 基类A的protected成员(函数)，成为派生类B的protected成员(函数)，与派生类B的protected成员(函数)融为一体。派生类B的成员函数可以直接访问基类A的保护成员(函数)，但派生类B类外的函数不能直接访问基类A的保护成员(函数)。
- 基类A的public成员(函数)，成为派生类B的public成员(函数)，与派生类B的public成员(函数)融和成一体。派生类B类外可以直接访问。

18

```

#include <iostream> // 保护成员继承的例子
using namespace std;
class A {
public:
    A(int a,int b) : a(a), b(b){ }
    void print(){cout << a << endl; }
    int b; // b为A类的公有成员
protected:
    int a; // a为A类的保护成员
};
class B : public A { // B类以公有方式继承A类
public:
    B(int a, int b, int c) : A(a, b), c(c){ }
    void print()
    { cout << c << endl; // 成员函数可直接访问本类的保护成员
      cout << b << endl; // 成员函数可直接访问基类的公有成员
      cout << a << endl; // 成员函数可直接访问基类的保护成员
    }
protected:
    int c; // c为B类的保护成员
};

```

19

```

class C : public B { // C类以公有方式继承B类
public:
    C(int a, int b, int c, int d) : B(a, b, c), d(d){ }
    void print()
    { cout << d << endl; // 成员函数可直接访问本类的保护成员
      cout << c << endl; // 成员函数可直接访问基类B的保护成员
      cout << b << endl; // 成员函数可直接访问基类A的保护成员
      cout << a << endl; // 成员函数可直接访问基类A的公有成员
    }
protected:
    int d; // d为C类的保护成员
};
void main()
{ C c(1,2,3,4);
  c.print();
  c.B::print();
  c.A::print(); // 也可以写为: c.B::A::print();
  cout << c.b << endl; // 可以直接访问基类的公有成员
  // cout << c.a << endl; // error C2248: "A::a": 无法访问 protected 成员(在 "A"类中声明)
  // cout << c.c << endl; // error C2248: "B::c": 无法访问 protected 成员(在 "B"类中声明)
  // cout << c.d << endl; // error C2248: "C::d": 无法访问 protected 成员(在 "C"类中声明)
}

```

运行结果:

```

4
3
2
1
3
2
1
1
1
2
请按任意键继续...

```

20

### 5.3.1 派生类中成员的访问权限

- 派生类中的成员函数可直接访问本派生类的公有成员、保护成员、私有成员
- 派生类中的成员函数可直接访问基类的公有成员、保护成员，但不可以直接访问基类的私有成员，需要通过调用基类的成员函数间接访问基类的私有成员
- 因此对于派生类，存在**四种访问权限的成员**：公有成员、保护成员、私有成员、不可直接访问的基类私有成员

21

### 5.3.1 派生类中成员的访问权限（续）

对于派生类之外的函数，若要访问派生类的成员：

- ✓可以直接访问所有(派生类和基类的)公有成员
- ✓可以通过调用派生类的成员函数访问所有(派生类和基类的)保护成员和派生类的私有成员
- ✓可以通过调用基类的公有成员函数，或通过调用派生类的成员函数间接调用基类的成员函数最终访问基类的私有成员

22

## 5.4 友元与继承

由于派生类不能直接访问基类中的私有成员，所以若一个派生类要直接访问基类中的私有成员，可以将这个派生类声明为基类的友元。

```
#include <iostream>
using namespace std;
class MyClass
{
    friend class YourClass;
    public: void Set(int x, int y);
    private: int X, Y;
};
class YourClass : public MyClass
{
    public: void Set(int x, int y, int val);
           void Print( );
    private: int val;
};
```

23

## 5.4 友元与继承(续1)

```
void MyClass::Set(int x, int y)
{ X=x; Y=y; }
void YourClass::Set(int x, int y, int v)
{ X=x; Y=y; // 可以直接访问基类的私有成员X,Y
  // MyClass::Set(x, y); //也可以调用基类的公有函数访问x,y
  val=v;
}
void YourClass::Print( )
{ cout <<X <<' ' <<Y <<' ' <<val <<endl; }
  // 可以直接访问并打印基类的私有成员X,Y
void main( )
{ YourClass Obj;
  Obj.Set(1, 3, 5);
  Obj.Print();
}
```

运行结果：  
1 3 5  
请按任意键继续...

24

## 5.5 访问权限调整

- ◆ 在私有继承的情况下，基类的所有成员和成员函数在派生类中具有私有成员的特性。
- ◆ 在保护继承的情况下，基类的所有共有成员和共有成员函数在派生类中具有保护成员的特性。
- ◆ 通过访问权限调整，可恢复派生类所继承的成员在基类中的访问权限。

```
class MyClass {  
    public: int f();  
           void f(int i);  
           void g();  
    protected: void h();  
    private: int X;  
};
```

25

## 5.5 访问权限调整(续1)

```
class YourClass : private MyClass {  
    public: void k();  
           void hh();  
           MyClass::f; // 恢复f的公有属性  
           MyClass::g; // 恢复g的公有属性  
    protected: MyClass::h; // 恢复h的保护属性  
};
```

**注意：**只能恢复原状，不能把private, protected成员的访问权限调整为 public 或 protected。

26

```

#include <iostream>
using namespace std;
class MyClass {
    public: int f( );
           void f(int i);
           void g( );
    protected: void h( );
    private: int X;
};
class YourClass : private MyClass {
    public: void k( );
           void hh( );
           MyClass::f; // 恢复f的公有属性
           MyClass::g; // 恢复g的公有属性
    protected: MyClass::h; // 恢复h的保护属性
};
int MyClass::f( )
{ return X; }

```

27

```

void MyClass::f(int i)
{ X=i; }
void MyClass::g( )
{ cout << X <<endl; }
void MyClass::h( )
{ X++; }
void YourClass::k( )
{ f( f( ) - 1);} // 成员函数k不能直接访问X，只能调用基类的函数间接访问
void YourClass::hh( )
{ h( ); } // h是保护函数，外界不能直接访问，只能由本类的成员函数访问
void main( )
{ YourClass b;
  b.f(5); b.g( ); // 初值置为 5
  b.hh( ); b.g( ); // 加1 写 b.h( );是不可以的
  b.k( ); b.g( ); // 减1
}

```

运行结果：  
5  
6  
5  
请按任意键继续...

28

## 5.6 成员名限定

在派生类和基类中可以声明同名的成员(函数)，不特别说明，在派生类中访问时，编译器会默认是派生类中的（后出现的），除非用：

基类名::基类成员名

进行成员名限定，才能调用同名的基类成员(函数)

```
#include <iostream>
using namespace std;
class MyClass {
    public: MyClass(int x);
           void Print();
    private: int X;
};
```

29

## 5.6 成员名限定(续1)

```
class YourClass : public MyClass {
    public: YourClass(int x, int y);
           void Print( ); //派生类的同名成员函数
    private: int X;    //派生类的同名私有成员
};

void MyClass ::Print( ) // 基类的Print函数
{ cout << X << " base class\n";}

void YourClass::Print( ) // 派生类的Print函数
{ MyClass::Print( ); // 派生类调用基类的Print必须加前缀
    // MyClass::, 否则成为自递归函数调用，会出现运行错误或
    // 没有运行结果而程序终止，编译时出现警告性错误：
    //warning C4717: "YourClass::Print": 如递归所有控件路径，函数将导致运行时堆栈溢出
    cout << X << " derivation class\n";
}
```

30



## 5.6 成员名限定(续2)

```
MyClass::MyClass(int x)
{ X=x;}
YourClass::YourClass(int x, int y) : MyClass(x) //构造基类
{ X=y;} // 注意：这里不能直接函数调用MyClass(x),
        // 构造函数不能在程序段{ }中被显式调用
void main( )
{   YourClass r(3,5);
    r.Print( );           // 调用派生类的Print函数
    r.MyClass::Print();   // 调用基类的Print函数
}
```

运行结果：  
3 base class  
5 derivation class  
3 base class  
请按任意键继续. . .

31

5.2节例子的另外一种写法（基类函数与派生类函数同名）：

```
#include <iostream>
using namespace std;
class Shape {
    public: void Set(int x, int y);
           void Move(int x, int y);
           void Display( ) const;
    private: int X, Y;
};
class Rectangle: public Shape {
    public: void Set(int x, int y, int w, int h);
           void Move(int x, int y);
           void Display( ) const;
    private: int Width, Height;
};
void Shape::Set(int x, int y)
{ X=x; Y=y; }
void Shape::Move(int x, int y)
{ X += x; Y += y; }
```

32

```

void Shape::Display( ) const
{ cout << X << ' ' << Y << endl; }
void Rectangle::Set(int x, int y, int w, int h)
{   Shape :: Set(x, y); // 这里必须加限定名Shape ::
    Width = w;   Height = h;
}
void Rectangle::Move(int x, int y)
{   Shape :: Move(x, y); // 这里必须加限定名Shape ::
}
void Rectangle::Display( ) const
{   Shape :: Display( ); // 这里必须加限定名Shape ::
    cout << ' ' << Width << ' ' << Height << endl;
}
void main( )
{   Rectangle R; R.Set(5,10,12,14); R.Display( );
    R.Move(+1,-1); R.Display( );
    Shape S;   S.Set(2, 4);   S.Display( );
}

```

运行结果:

```

5 10
12 14
6 9
12 14
2 4
请按任意键继续. . .

```

33

如果Rectangle::Set( )写为:

```

void Rectangle::Set(int x, int y, int w, int h)
{   Set(x, y); // 这里不加限定名Shape ::
    Width = w;   Height = h;
}

```

编译结果:

error C2660: "Rectangle::Set": 函数不接受2 个参数

如果Rectangle::Display( ) const 写为:

```

void Rectangle::Display( ) const
{   Display( ); // 这里不加限定名Shape ::
    cout << ' ' << Width << ' ' << Height << endl;
}

```

编译结果:

warning C4717: "Rectangle::Display": 如递归所有控件路径, 函数将导致运行时堆栈溢出  
运行结果出错: 或者无任何输出而结束程序运行。



34

这里： `void Shape::Display( ) const`  
和 `void Rectangle::Display( ) const`  
不是重载函数，因为两个函数一模一样，而且不是同一个类的。

同样： `void Shape::Set(int x, int y)`  
和 `void Rectangle::Set(int x, int y, int w, int h)`  
也不是重载函数，因为他们不是同一个类的。

**注意：**

派生类的同名函数**掩蔽**了基类的同名函数，除非加限定名 **Shape ::** 指明要使用基类的同名函数，否则在编译器看来是直接自递归调用派生类的函数本身，造成要么编译错误，要么运行错误（无条件直接自递归调用）。

35

## 5.7 派生类的构造函数与复制构造函数

- ◆ 派生类的构造函数与复制构造函数如何写？

若B是基类，A是派生类，

- ◆ 则派生类A的构造函数的书写格式为：

```
A::A(参数表): B(参数) // 先构造基类无名对象B
{ .....
}
```

- ◆ 派生类A的复制构造函数的书写格式为：

```
A::A(const A &p): B(p) // 注意：这里B的参数为A类对象p,
// 用对象p中基类部分去复制构造无名基类对象B
{ .....
}
```

36

```

#include <iostream>
using namespace std;
class Shape {
    public: Shape(int x=0, int y=0);
           Shape(const Shape &p);
           ~Shape( );
           void Set(int x, int y);
           void Move(int x, int y);
           void Display( ) const;
    private: int x, y;
};
Shape::Shape(int x, int y) : x(x), y(y)
{   cout << "Shape constructor "; Display( ); }
Shape::Shape(const Shape &p) : x(p.x), y(p.y)
{   cout << "Shape copy constructor "; Display( ); }
Shape::~Shape( )
{   cout << "Shape Destructor "; Display( ); }

```

37

```

void Shape::Set(int x, int y)
{ this->x=x; this->y=y; }
void Shape::Move(int x, int y)
{ this->x += x; this->y += y; }
void Shape::Display( ) const
{ cout << x << ' ' << y << endl; }
class Rectangle: public Shape {
    public: Rectangle(int x=0, int y=0, int w=0, int h=0);
           Rectangle(const Rectangle &p);
           ~Rectangle( );
           void Set(int x, int y, int w, int h);
           void Move(int x, int y);
           void Display( ) const;
    private: int Width, Height;
};

```

38

```

Rectangle::Rectangle(int x, int y, int w, int h) : Shape(x,y)
{ Width=w; Height=h;
  cout << "Rectangle constructor "; Display( );
}
Rectangle::Rectangle(const Rectangle &p) : Shape(p)
{ Width=p.Width; Height =p. Height;
  cout << "Rectangle copy constructor "; Display( );
}
Rectangle::~Rectangle()
{ cout << "Rectangle Destructor "; Display( );
}
void Rectangle::Set(int x, int y, int w, int h)
{ Shape::Set(x, y); // 不可以 this->x = x; this->y = y;
  Width = w; Height = h;
}

```

39

```

void Rectangle::Move(int x, int y)
{ Shape::Move(x, y); // 不可以 this->x += x; this->y += y;
}
void Rectangle::Display( ) const
{ Shape::Display(); // 不可以 cout <<x<<','<<y<<endl;
  cout << ' ' << Width << ' ' << Height << endl;
}
void main( )
{ Shape S; S.Display( );
  S.Set(2, 4); S.Display( );
  S.Move(1, 1); S.Display( );
  Rectangle R(1, 1, 10, 10); R.Display( );
  R.Set(5, 10, 12, 14); R.Display( );
  R.Move(-1, -2); R.Display( );
  Rectangle T(R); T.Display( );
  T.Move(+4, +4); T.Display( );
}

```

40

运行结果:  
 Shape constructor 0,0  
 0,0  
 2,4  
 3,5  
 Shape constructor 1,1  
 Rectangle constructor 1,1  
 10 10  
 1,1  
 10 10  
 5,10  
 12 14  
 4,8  
 12 14  
 Shape copy constructor 4,8  
 Rectangle copy constructor 4,8  
 12 14  
 4,8  
 12 14  
 8,12  
 12 14  
 Rectangle Destructor 8,12  
 12 14  
 Shape Destructor 8,12  
 Rectangle Destructor 4,8  
 12 14  
 Shape Destructor 4,8  
 Shape Destructor 3,5  
 请按任意键继续...

```

C:\Windows\system32\cmd.exe
Shape constructor 0,0
0,0
2,4
3,5
Shape constructor 1,1
Rectangle constructor 1,1
10 10
1,1
10 10
5,10
12 14
4,8
12 14
Shape copy constructor 4,8
Rectangle copy constructor 4,8
12 14
4,8
12 14
8,12
12 14
Rectangle Destructor 8,12
12 14
Shape Destructor 8,12
Rectangle Destructor 4,8
12 14
Shape Destructor 4,8
Shape Destructor 3,5
请按任意键继续...
  
```

41

如果把派生类Rectangle的构造函数、复制构造函数  
 改写为:

```

Rectangle::Rectangle(int x, int y, int w, int h) //: Shape(x,y)
{ Width=w; Height=h;
  cout << "Rectangle constructor ";
  Shape::Display();
  cout << Width << ',' << Height << endl;
}
Rectangle::Rectangle(const Rectangle &p) // : Shape(p)
{ Width=p.Width; Height =p. Height;
  cout << "Rectangle copy constructor ";
  Shape::Display();
  cout << Width << ',' << Height << endl;
}
  
```

不调用基类的(复制)构造函数, 运行结果会是什么?

42

运行结果:

```
Shape constructor 0,0
0,0
2,4
3,5
Shape constructor 0,0
Rectangle constructor 0,0
10 10
0,0
10 10
5,10
12 14
4,8
12 14
Shape constructor 0,0
Rectangle copy constructor 0,0
12 14
0,0
```

```
12 14
4,4
12 14
Rectangle Destructor 4,4
12 14
Shape Destructor 4,4
Rectangle Destructor 4,8
12 14
Shape Destructor 4,8
Shape Destructor 3,5
请按任意键继续...
```

Rectangle的构造函数、复制构造函数通过调用缺省参数的Shape构造函数构造了基类Shape, 但Rectangle的构造函数没有给基类的Shape赋值1,1。而Rectangle的复制构造函数也没有真正复制p的Shape值给新的对象。

43

上面的程序中如果基类Shape的构造函数去掉缺省值:

将 Shape(int x=0, int y=0);

改为: Shape(int x, int y);

编译结果为:

test.cpp(34) : error C2512: "Shape": 没有合适的默认构造函数可用

test.cpp(38) : error C2512: "Shape": 没有合适的默认构造函数可用

test.cpp(56) : error C2512: "Shape": 没有合适的默认构造函数可用

这是指程序中:

```
Rectangle::Rectangle(int x, int y, int w, int h) //: Shape(x,y)
```

```
Rectangle::Rectangle(const Rectangle &p) // : Shape(p)
```

```
Shape S;
```

这三行需要调用Shape缺省值的默认构造函数, 因此无法通过编译。

44

派生类Rectangle的构造函数、复制构造函数还可以写为:

```
Rectangle::Rectangle(int x, int y, int w, int h)
    : Shape(x,y) , Width(w), Height(h)
{ cout << "Rectangle constructor "; Display();
}
Rectangle::Rectangle(const Rectangle &p) : Shape(p) ,
    Width(p.Width), Height(p.Height)
{ cout << "Rectangle copy constructor "; Display();
}
```

从理论上讲, 这样书写运行效率更高一些。是在构造Width 和 Height时, 同时赋初值。

注意: Shape(p) 不要写为: Shape(&p)

45

派生类Rectangle的构造函数、复制构造函数不能写为:

```
Rectangle::Rectangle(int x, int y, int w, int h)
{ Shape(x,y);
  Width=w; Height=h;
  cout << "Rectangle constructor "; Display();
}
Rectangle::Rectangle(const Rectangle &p)
{ Shape(p);
  Width=p.Width; Height =p.Height;
  cout << "Rectangle copy constructor "; Display();
}
```

这样Rectangle复制构造函数编译会指示Shape(p); 行有错误:  
error C2082: 形参“p”的重定义

改为强制类型转换:

```
Rectangle::Rectangle(const Rectangle &p)
{ Shape((Shape&)p);
  Width=p.Width; Height =p.Height;
  cout << "Rectangle copy constructor "; Display();
}
```

46



```

运行结果:
Shape constructor 0,0
0,0
2,4
3,5
Shape constructor 0,0
Shape constructor 1,1
Shape Destructor 1,1
Rectangle constructor 0,0
10 10
0,0
10 10
5,10
12 14
4,8
12 14
Shape constructor 0,0
Shape copy constructor 4,8
Shape Destructor 4,8
Rectangle copy constructor 0,0
12 14
0,0
12 14
4,4
12 14
Rectangle Destructor 4,4
12 14
Shape Destructor 4,4
Rectangle Destructor 4,8
12 14
Shape Destructor 4,8
Shape Destructor 3,5
请按任意键继续...

```

```

原来正确的运行结果:
Shape constructor 0,0
0,0
2,4
3,5
Shape constructor 1,1
Rectangle constructor 1,1
10 10
1,1
10 10
5,10
12 14
4,8
12 14
Shape copy constructor 4,8
Rectangle copy constructor 4,8
12 14
4,8
12 14
8,12
12 14
Rectangle Destructor 8,12
12 14
Shape Destructor 8,12
Rectangle Destructor 4,8
12 14
Shape Destructor 4,8
Shape Destructor 3,5
请按任意键继续...

```

47

分析:

1. Rectangle的构造函数、复制构造函数通过调用缺省值的Shape构造函数构造了Rectangle对象中的Shape基类部分，但Rectangle的构造函数没有给Rectangle对象中的Shape基类赋值(1,1)，而Rectangle的复制构造函数也没有真正复制p的Shape值(4,8)给新的Rectangle对象中的Shape基类。
2. Rectangle构造函数中的Shape(x,y); 实际上是构造了一个初值为(1,1)的临时Shape无名对象，而且构造完成后，因为没什么用处，立刻就被析构掉了。
3. Rectangle复制构造函数中的Shape((Shape&)p);实际上是复制构造了一个初值为(4,8)的临时Shape无名对象，而且构造完成后，因为没什么用处，立刻也被析构掉了。

上例的基类private成员如果改为protected成员:

48

把私有成员改为保护成员后的例子：

```
#include <iostream>
using namespace std;
class Shape {
    public: Shape(int x=0, int y=0);
           Shape(const Shape &p);
           ~Shape( );
           void Set(int x, int y);
           void Move(int x, int y);
           void Display( ) const;
    protected: int x, y;
};
Shape::Shape(int x, int y)
{ this->x = x; this->y = y;
  cout << "Shape constructor "; Display( );
}
Shape::Shape(const Shape &p)
{ this->x=p.x; this->y=p.y;
  cout << "Shape copy constructor "; Display( );
}
```

49

```
Shape::~~Shape( )
{ cout << "Shape Destructor "; Display( ); }
void Shape::Set(int x, int y)
{ this->x=x; this->y=y; }
void Shape::Move(int x, int y)
{ this->x += x; this->y += y; }
void Shape::Display( ) const
{ cout << x << ',' << y << endl; }
class Rectangle: public Shape {
    public: Rectangle(int x=0, int y=0, int w=0, int h=0);
           Rectangle(const Rectangle &p);
           ~Rectangle( );
           void Set(int x, int y, int w, int h);
           void Move(int x, int y);
           void Display( ) const;
    protected: int Width, Height;
};
```

50

```

Rectangle::Rectangle(int x, int y, int w, int h) : Shape(x,y),
    Width(w), Height(h)
{ cout << "Rectangle constructor ";
  Display( );
}
Rectangle::Rectangle(const Rectangle &p) : Shape(p),
    Width(p.Width), Height(p.Height)
{ cout << "Rectangle copy constructor ";
  Display( );
}
Rectangle::~Rectangle( )
{ cout << "Rectangle Destructor "; Display( );
}
void Rectangle::Set(int x, int y, int w, int h)
{ this->x = x; this->y = y; // 可以直接访问基类保护成员
  Width = w; Height = h;
}

```

51

```

void Rectangle::Move(int x, int y)
{ this->x += x; this->y += y; // 可以直接访问基类保护成员
}
void Rectangle::Display( ) const
{ cout<<' '<<x <<' '<<y <<endl // 可以直接输出基类保护成员
  <<' '<< Width <<' '<< Height << endl;
}
void main( )
{ Shape S; S.Display( );
  S.Set(2, 4); S.Display( );
  S.Move(1, 1); S.Display( );
  Rectangle R(1, 1, 10, 10); R.Display( );
  R.Set(5, 10, 12, 14); R.Display( );
  R.Move(-1, -2); R.Display( );
  Rectangle T(R); T.Display( );
  T.Shape::Move(+4, +4); T.Display( );
}

```

52

运行结果:

```
Shape constructor 0,0
0,0
2,4
3,5
Shape constructor 1,1
Rectangle constructor 1 1
10 10
1 1
10 10
5 10
12 14
4 8
12 14
Shape copy constructor 4,8
Rectangle copy constructor 4 8
```

```
12 14
4 8
12 14
8 12
12 14
Rectangle Destructor 8 12
12 14
Shape Destructor 8,12
Rectangle Destructor 4 8
12 14
Shape Destructor 4,8
Shape Destructor 3,5
请按任意键继续...
```

53

说明:

本例中Rectangle的复制构造函数: //调用基类复制构造函数  
Rectangle::Rectangle(const Rectangle &p) : Shape(p),  
Width(p.Width), Height(p.Height)  
{ cout << "Rectangle copy constructor ";  
Display( );  
}

也可以写成(调用基类Shape的构造函数):  
Rectangle::Rectangle(const Rectangle &p) : Shape(p.x, p.y),  
Width(p.Width), Height(p.Height)  
{ cout << "Rectangle copy constructor ";  
Display( );  
}

因为Rectangle的成员函数可以直接访问基类的保护成员。

54

但不能写成:

```
Rectangle::Rectangle(const Rectangle &p) : x(p.x), y(p.y),  
    Width(p.Width), Height(p.Height)  
{ cout << "Rectangle copy constructor ";  
    Display( );  
}
```

因为x和y是基类成员，只能由基类的构造函数或复制构造函数来构造。 写成:

```
Rectangle::Rectangle(const Rectangle &p)  
{ x= p.x; y=p.y;  
    Width=p.Width; Height=p.Height;  
    cout << "Rectangle copy constructor ";  
    Display( );  
}
```

还勉强算可以，是隐含先用缺省值Shape构造函数构造Shape，(如果Shape构造函数没有缺省值，编译时将出错)再构造Width和Height后，再逐个赋值(两次赋值影响效率)。

55

## 5.8 带基类内嵌对象成员的派生类

若有类Point，私有成员为(X,Y)，若要以Point作为基类，生成派生类Line，Line中希望有两个Point对象(用两个点来表示平面坐标上的一条直线)，若想派生两次Point是不允许的，写为:

```
class Line: public Point, public Point
```

编译结果是:

error C2500: "Line": "Point"已是直接基类

因此只能派生一次Point作为基类，另设一个Point内嵌对象作为Line的私有成员:

```
class Line: public Point {  
    public: .....  
    private: Point p2;  
}
```

那么如何写Line的构造函数、复制构造函数及成员函数呢?

56

```

#include <iostream>
using namespace std;
class Point {
    public: Point(int x=0, int y=0);
           Point(const Point &p);
           ~Point( );
           void Display( ) const;
    private: int X, Y;
};
Point::Point(int x, int y) : X(x), Y(y)
{ cout << "Point constructor " ; Display( ); }
Point::Point(const Point &p) : X(p.X), Y(p.Y)
{ cout << "Point copy constructor "; Display( ); }
Point::~~Point( )
{ cout << "Point Destructor "; Display( ); }
void Point ::Display( ) const
{ cout << X << ' ' << Y << endl; }

```

57

```

class Line: public Point {
    public: Line(int x1=0, int y1=0, int x2=0, int y2=0);
           Line(Point &p1, Point &p2);
           Line(const Line &p);
           ~Line( );
           void Display( ) const;
    private: Point p2;
};
Line::Line(int x1, int y1, int x2, int y2) : Point(x1,y1), p2(x2,y2)
{ cout << "Line constructor "; Display( ); }
Line::Line(Point &p1, Point &p2) : Point(p1), p2(p2)
{ cout << "Line constructor2 "; Display( ); }
Line::Line(const Line &p) : Point(p), p2(p.p2)
{ cout << "Line copy constructor "; Display( ); }
Line::~~Line( )
{ cout << "Line Destructor "; Display( ); }
void Line ::Display( ) const
{ Point::Display( ); p2.Display( ); }

```

58

```

void main( )
{ Point p1(1, 2), p2(3, 4); // 构造两个点
  p1.Display( ); p2.Display( );
  Line L1(-1,-1,5,6); // 用两对坐标构造一条线
  L1.Display( );
  Line L2(p1, p2); // 用两个点构造一条线
  L2.Display( );
  Line L3(L2); // 用一条线复制构造另一条线
  L3.Display( );
}

```

注意派生类Line的构造函数、复制构造函数的写法：

```
Line::Line(int x1, int y1, int x2, int y2) : Point(x1,y1), p2(x2,y2)
```

```
Line::Line(Point &p1, Point &p2) : Point(p1), p2(p2)
```

```
Line::Line(const Line &p) : Point(p), p2(p.p2)
```

先直接用Line对象p构造当前对象的基类无名对象Point，再用Line对象p的内嵌对象p.p2构造当前对象的内嵌对象p2。

而Display函数是先调用基类的Display，再调用内嵌对象p2的Display分别显示Line中两个Point对象的坐标。

59

运行结果为：

```

Point constructor 1,2 // 构造p1
Point constructor 3,4 // 构造p2
1,2
3,4
Point constructor -1,-1 // 构造基类
Point constructor 5,6 // 构造p2
Line constructor -1,-1 // 构造L1
5,6
-1,-1
5,6
Point copy constructor 1,2 // 复制构造基类
Point copy constructor 3,4 // 复制构造p2
Line constructor2 1,2 // 构造L2
3,4
1,2
3,4
Point copy constructor 1,2 // 复制构造基类
Point copy constructor 3,4 // 复制构造p2
Line copy constructor 1,2 // 复制构造L3

```

```

3,4
1,2
3,4
Line Destructor 1,2 // 析构L3
3,4
Point Destructor 3,4 // 析构p2
Point Destructor 1,2 // 析构基类
Line Destructor 1,2 // 析构L2
3,4
Point Destructor 3,4 // 析构p2
Point Destructor 1,2 // 析构基类
Line Destructor -1,-1 // 析构L1
5,6
Point Destructor 5,6 // 析构p2
Point Destructor -1,-1 // 析构基类
Point Destructor 3,4 // 析构p2
Point Destructor 1,2 // 析构p1
请按任意键继续...

```

60

如果把内嵌对象的成员改为protected，派生类是否可以直接访问内嵌对象的成员呢？

```
#include <iostream>
using namespace std;
class Point {
public:
    Point(int x=0, int y=0);
    Point(const Point &p);
    ~Point( );
    void Display( ) const;
protected:
    int X, Y;
};
Point::Point(int x, int y) : X(x), Y(y)
{ cout << "Point constructor "; Display( ); }
Point::Point(const Point &p) : X(p.X), Y(p.Y)
{ cout << "Point copy constructor "; Display( ); }
Point::~~Point( )
{ cout << "Point Destructor "; Display( ); }
void Point ::Display( ) const
{ cout << X << ',' << Y << endl; }
```

61

```
class Line: public Point {
public:
    Line(int x1=0, int y1=0, int x2=0, int y2=0);
    Line(Point &p1, Point &p2);
    Line(const Line &p);
    ~Line( );
    void Display( ) const;
protected:
    Point p2;
};
Line::Line(int x1, int y1, int x2, int y2) : Point(x1,y1), p2(x2,y2)
{ cout << "Line constructor "; Display( ); }
Line::Line(Point &p1, Point &p2) : Point(p1), p2(p2)
{ cout << "Line constructor2 "; Display( ); }
Line::Line(const Line &p) : Point(p), p2(p.p2)
{ cout << "Line copy constructor "; Display( ); }
Line::~~Line( )
{ cout << "Line Destructor "; Display( ); }
```

62



```

void Line::Display( ) const
{ cout << X << ',' << Y << endl;          // Point::Display( );
  // 可以直接访问基类的保护成员
  cout << p2.X << ',' << p2.Y << endl; // p2.Display( );
  // 编译错误,不可以直接访问内嵌对象的保护成员,只能间接访问
}
void main( )
{ Point p1(1, 2), p2(3, 4); // 构造两个点
  p1.Display( ); p2.Display( );
  Line L1(-1,-1,5,6); // 用两对坐标构造一条线
  L1.Display( );
  Line L2(p1, p2); // 用两个点构造一条线
  L2.Display( );
  Line L3(L2); // 用一条线复制构造另一条线
  L3.Display( );
}

```

63

编译结果:

1> test.cpp(36) : error C2248: “Point::X”: 无法访问 protected 成员  
(在 “Point”类中声明)

1> test.cpp(8) : 参见 “Point::X”的声明

1> test.cpp(3) : 参见 “Point”的声明

1> test.cpp(36) : error C2248: “Point::Y”: 无法访问 protected 成员  
(在 “Point”类中声明)

1> test.cpp(8) : 参见 “Point::Y”的声明

1> test.cpp(3) : 参见 “Point”的声明

结论: 派生类成员函数可以直接访问其基类的保护成员,  
但不能直接访问与基类同类型的内嵌对象的保护成员。  
只能通过调用内嵌对象的公有成员函数间接访问, 或者  
把派生类声明为基类的友元。

64

第7次作业：

见网络学堂第7次作业

前2题必做，第3题选做

65